

Midterm Report

Automated Product Inquiry Response System for Light Recycling Program



Student Name: Namesh Mathara Arachchi Vidanalage

Student ID: 300359798

Course: CSIS 4495 – Applied Research Project

Section: 002

Midterm Video Demonstration: https://youtu.be/r8-W6cK_NPw

Contents

Checklist	3
Introduction	3
Background and Context	3
Problem Statement and Research Questions	3
Existing Knowledge Gaps and Research Challenges	4
Relevant Literature and Research Foundations	4
Hypotheses, Assumptions, and Potential Benefits	5
Project Proposal Summary	6
Project Phases	6
Changes in the Project Implementation and Justifications	7
Reordering Project Phases	7
Modifications in Product Guide Data Extraction	7
Adjustments in PDF Data Extraction Approach	7
Change in Backend Data Storage Selection	8
Project Planning and Timeline	8
Implemented Features	9
Frontend – UI (member portal)	9
Frontend – UI (admin portal)	10
Frontend – Code	13
Backend – Code	15
Azure Blob Storage and Azure Table Storage	18
ML Model – Code	21
Product Upload – Code	22
Work Logs	24
Acknowledgements	24
References	24

Checklist

☒	GitHub Repository Code Check-In - Implementation Directory
☒	Video Demo of the Product
☒	Video Link - https://youtu.be/r8-W6cK_NPw
☒	Installation Instructions - PROJECT INSTALLATION INSTRUCTIONS MIDTERM.md
☒	Project Description - README.md
☒	Midterm Report Blackboard Submission
☒	GitHub Repository Midterm Report Check-in

Introduction

Background and Context

Product Care Association (PCA) is a leading industry-led organization dedicated to environmental protection through recycling programs for post-consumer products such as paint, household hazardous waste, lights, and alarms. This research focuses specifically on light recycling and aims to develop an automated Product Inquiry Response System to enhance operational efficiency.

PCA successfully diverted over 10.8 million light bulbs from landfills in 2022, operating across multiple Canadian provinces, including British Columbia, Manitoba, Ontario, Quebec, Nova Scotia, and Prince Edward Island. The organization collaborates with recycling centers, retail chains, and bottle depots to facilitate the recycling of lighting products. However, the current product inquiry process remains largely manual, leading to inefficiencies in response times, inconsistencies in decision-making, and increased operational workload.

Problem Statement and Research Questions

Before accepting a product for recycling, PCA members must determine whether it falls under the approved recycling program. This verification process involves consulting a Product Guide or manually searching through historical product inquiries stored in Excel sheets. The existing approach presents several inefficiencies:

- Repetitive inquiries from different members for the same product.
- Manual searches through extensive product records are time-consuming.

- Inconsistent decision-making, as different staff members handle inquiries differently.

This research seeks to automate and optimize the product inquiry process by answering the following key questions:

1. Can a hybrid Machine Learning (ML) model accurately match products to PCA's product database?
2. What degree of automation is achievable in reducing manual intervention while maintaining accuracy?
3. How can text and image data be effectively integrated to match products with high confidence?

Existing Knowledge Gaps and Research Challenges

While PCA maintains a Product Guide as the primary reference, its complex structure, inconsistent formatting, and lack of a standardized database present challenges for automation. My research into text and image extraction techniques has revealed that:

- The Product Guide does not follow a uniform tabular format, making direct extraction difficult.
- Merged cells, inconsistent labeling, and multiple images within single rows complicate automated data processing.
- Manual verification remains necessary to ensure data integrity, as fully automating extraction would require extensive custom handling.

Additionally, historical product inquiries stored in Excel sheets suffer from data integrity issues, including inconsistent category labeling, formatting errors, and missing values. For this reason, I have started the project by using the product guide to create the products table. Once the issues in the spreadsheet have been addressed, I can merge its data into the products table.

Relevant Literature and Research Foundations

This study builds upon existing research in Natural Language Processing (NLP), Computer Vision (CV), and database optimization for structured data retrieval. Key areas of investigation include:

- Text Processing: Utilizing BERT-based sentence embeddings for contextual product name/description matching.
- Image Processing: Employing CNN models (ResNet50/EfficientNet) to compare product images for similarity-based retrieval.

- **Hybrid Decision Models:** Combining text and image similarity scores to improve accuracy in classifying product inquiries.

Research has shown that NLP models like BERT excel at contextual understanding, making them ideal for product description comparisons. Likewise, CNN-based image recognition models can accurately classify product images, making them suitable for matching product inquiries based on visual similarities. This research aims to bridge the gap by integrating both approaches into a single AI-driven system.

Hypotheses, Assumptions, and Potential Benefits

Hypotheses and Assumptions

- A hybrid NLP + CV ML model will accurately match product inquiries to PCA's product database.
- Automating inquiries will reduce response times by at least 50%.
- Sufficient historical data exists to train an effective ML model.
- Product images provided in inquiries will be of reasonable quality for analysis.

Potential Benefits

This research will yield significant benefits for PCA and its recycling partners, including:

- **Operational Efficiency:** Automating the inquiry process will reduce manual workload, allowing staff to focus on higher-value tasks.
- **Faster Response Times:** Members will receive instantaneous responses, improving service levels.
- **Consistency in Decision-Making:** AI-driven classification will minimize human error and ensure uniform responses.
- **Scalability:** The system can be expanded to support other recyclable product categories beyond lighting.
- **Cost Savings:** Automating the process will reduce operational expenses linked to manual handling.
- **AI-Driven Innovation:** This project represents PCA's first ML-powered operational system, paving the way for future AI implementations.

Project Proposal Summary

This project aims to develop an AI-powered product inquiry system that automates the identification of recyclable products using machine learning and a web-based interface. The project consists of seven key phases, ensuring a structured and efficient implementation.

Project Phases

1. Defining the Project and Scope – Establishing project objectives, requirements, and key deliverables.
2. Product Guide Data Extraction – Extracting structured data from the product guide (PDF) and historical product inquiries sheets (Excel).
3. ML Model - Text Matching – Implementing BERT-based text processing for product name/description matching.
4. ML Model - Image Matching – Utilizing CNN models (ResNet50/EfficientNet) for image-based product identification.
5. System Integration & Testing – Combining all components and ensuring interoperability.
6. Web Portal Development – Building a Flask-based portal for product inquiries.
7. Deployment, Security & Final Testing – Securing, optimizing, and deploying the solution.

Phase 2: Product Guide Data Extraction

- Product Guide Data Extraction (PDF): PyMuPDF and Tesseract OCR will be used to extract structured product information from scanned PDFs. PyMuPDF enables parsing, while Tesseract OCR extracts text from images.
- Historical Inquiry Data Processing (Excel): Analyzing past inquiries stored in an Excel file using Pandas for data manipulation and NLP techniques for text normalization.

Phase 3 & 4: Machine Learning Model Development

- Text Matching: A BERT-based model will generate contextual embeddings for product descriptions, ensuring accurate text similarity detection.
- Image Matching: CNN models (ResNet50/EfficientNet) will extract image features and compare them with stored product images.
- Decision Model: An ensemble approach will combine text and image similarity scores, improving classification accuracy and robustness.

Phase 6: Web Portal Development

- **Frontend:** Developed using Flask and Bootstrap to provide a user-friendly and responsive interface.
- **Backend:** Flask will handle API requests and integrate machine learning models, while Azure SQL will store inquiry data.
- **Data Storage:** Images will be stored in Azure Blob Storage, while textual data will be managed in Azure Table Storage.

Changes in the Project Implementation and Justifications

Reordering Project Phases

- **Change:** Initially, the plan was to develop the ML model first and then build the web portal. However, the web portal has been developed first, and ML implementation will follow.
- **Justification:** Building the web portal first provided a clearer end-to-end perspective of the project. This approach also allows for early feedback from employees, ensuring that the portal aligns with business needs before integrating the ML model.

Modifications in Product Guide Data Extraction

- **Change:** Initially planned to use both the Product Guide (PDF) and past product inquiry data (Excel). Currently, only the Product Guide is being used due to data integrity issues in the spreadsheet.
- **Justification:** The Excel data contains inconsistencies that need to be addressed to maintain dataset quality for ML training. Since data integrity significantly impacts ML model performance, the focus has been on structuring the Product Guide first. The spreadsheet data will be integrated later once cleaned.

Adjustments in PDF Data Extraction Approach

- **Change:** Initially proposed automated extraction using PyMuPDF and Tesseract OCR. Now, data is being manually extracted and formatted in Excel using Adobe Acrobat Pro.
- **Justification:** The Product Guide has a complex table structure with merged cells, inconsistent labeling, and multiple images per row, making automated extraction unreliable. Given that the dataset has only 400 records, manual formatting is more efficient than

developing custom scripts to handle inconsistencies. This ensures higher accuracy with less effort.

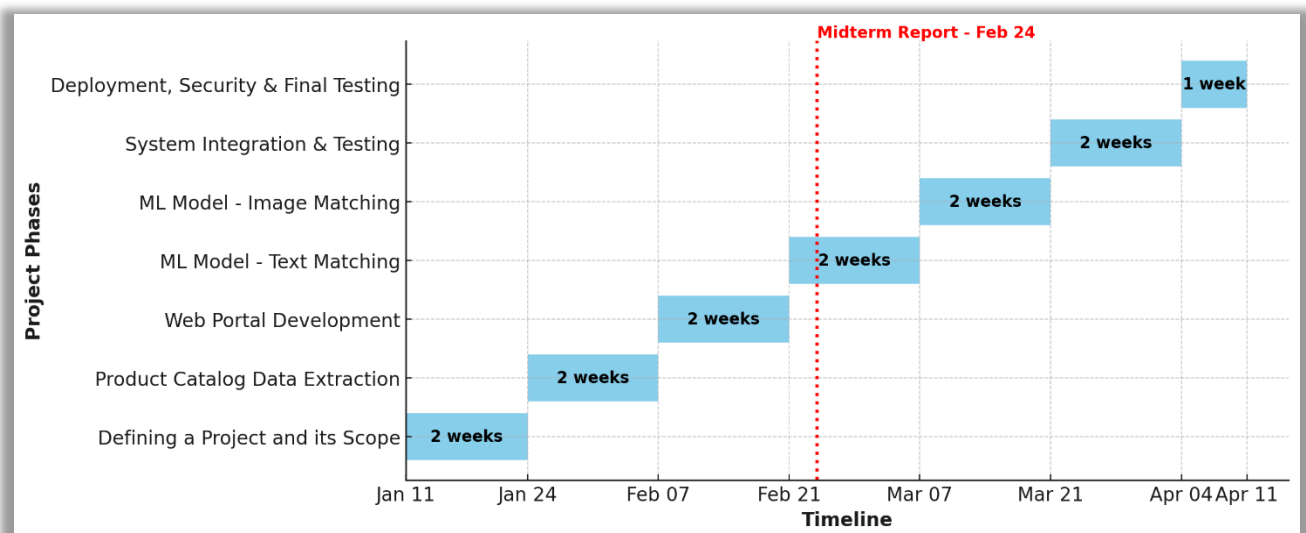
Change in Backend Data Storage Selection

- **Change:** Originally planned to store inquiry data in Azure SQL Server. Now, Azure Table Storage is being used instead.
- **Justification:** The data structure is simple, and the inquiry frequency is low, making Azure SQL Server unnecessary. Azure Table Storage is significantly more cost-effective, scalable, and sufficient for the current use case.

Project Planning and Timeline

As mentioned in the previous section, the only modification is the decision to develop the Web Portal before the ML model. Other than this adjustment, the timeline remains unchanged, including the time allocated for each phase. See below updated timeline and Gantt chart.

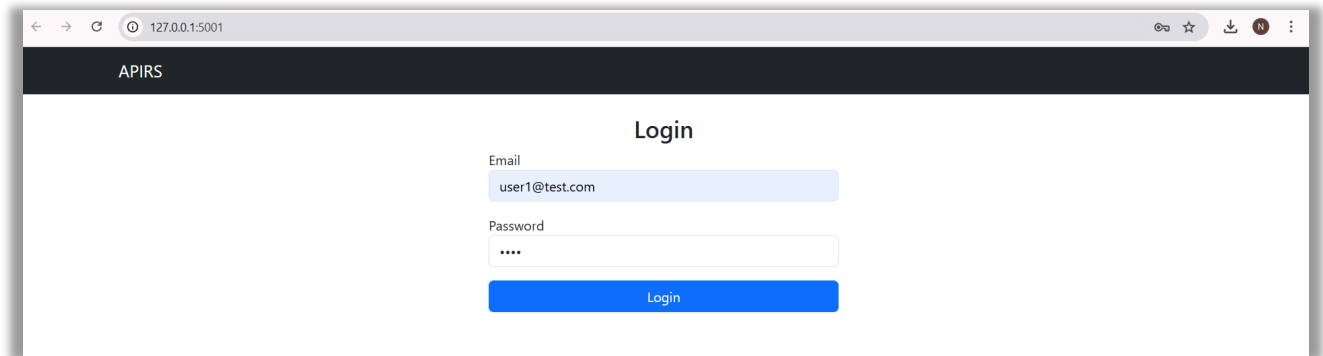
Phase	Start Date	End Date	Duration (Weeks)
Defining a Project and its Scope		DONE	2
Product Catalog Data Extraction		DONE	2
Web Portal Development		DONE	2
ML Model - Text Matching	2025-02-21	2025-03-07	2
ML Model - Image Matching	2025-03-07	2025-03-21	2
System Integration & Testing	2025-03-21	2025-04-04	2
Deployment, Security & Final Testing	2025-04-04	2025-04-11	1



Implemented Features

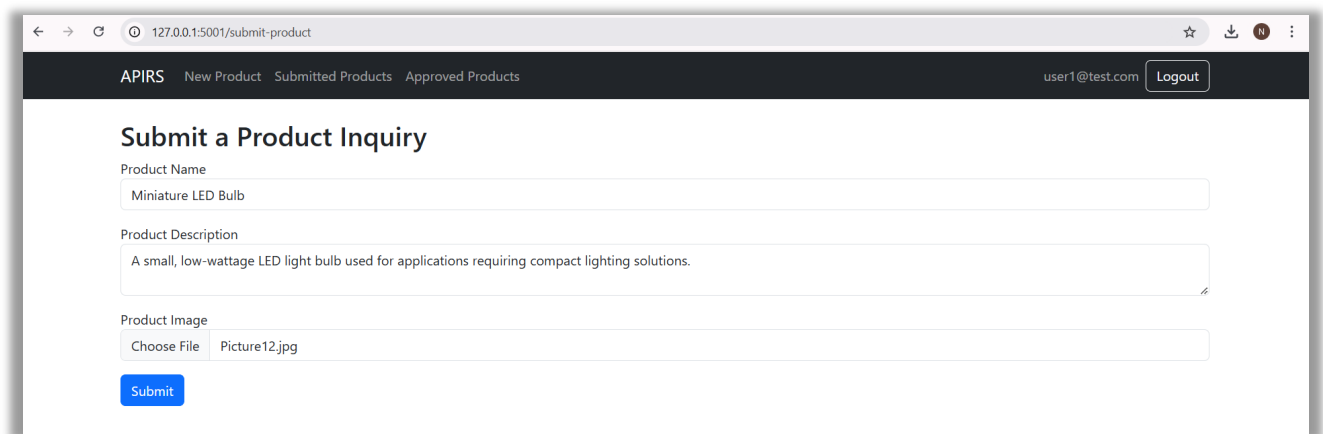
Frontend – UI (member portal)

Member Login Page



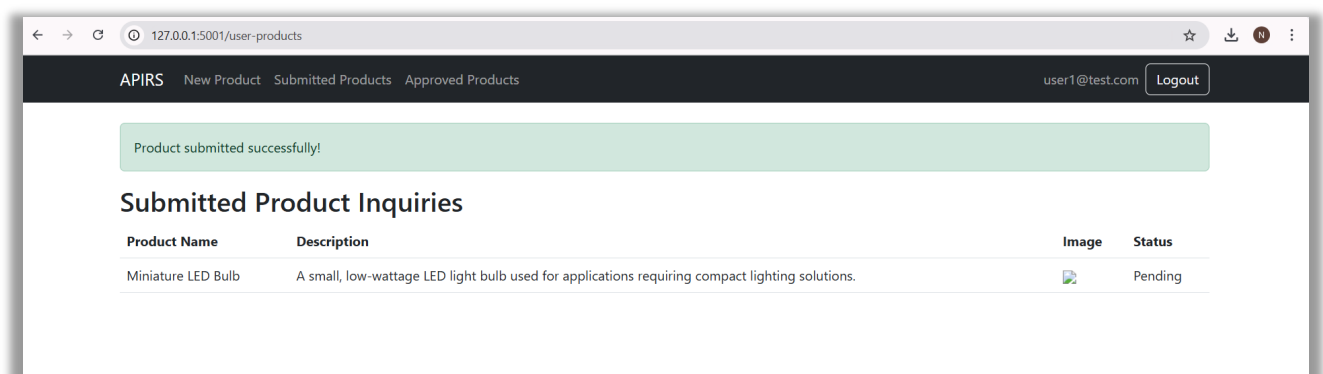
A screenshot of a web browser showing the login page of the APIRS member portal. The browser's address bar displays '127.0.0.1:5001'. The page has a dark header with the text 'APIRS'. The main content area is white and features a 'Login' heading. Below the heading are two input fields: 'Email' with the value 'user1@test.com' and 'Password' with masked characters '****'. A blue 'Login' button is positioned below the password field.

New Product Page



A screenshot of the 'Submit a Product Inquiry' page in the APIRS member portal. The browser address bar shows '127.0.0.1:5001/submit-product'. The dark header includes the 'APIRS' logo and navigation links: 'New Product', 'Submitted Products', and 'Approved Products'. On the right, it shows the user 'user1@test.com' and a 'Logout' button. The main form contains three sections: 'Product Name' with the text 'Miniature LED Bulb', 'Product Description' with the text 'A small, low-wattage LED light bulb used for applications requiring compact lighting solutions.', and 'Product Image' with a 'Choose File' button and the filename 'Picture12.jpg'. A blue 'Submit' button is at the bottom left of the form.

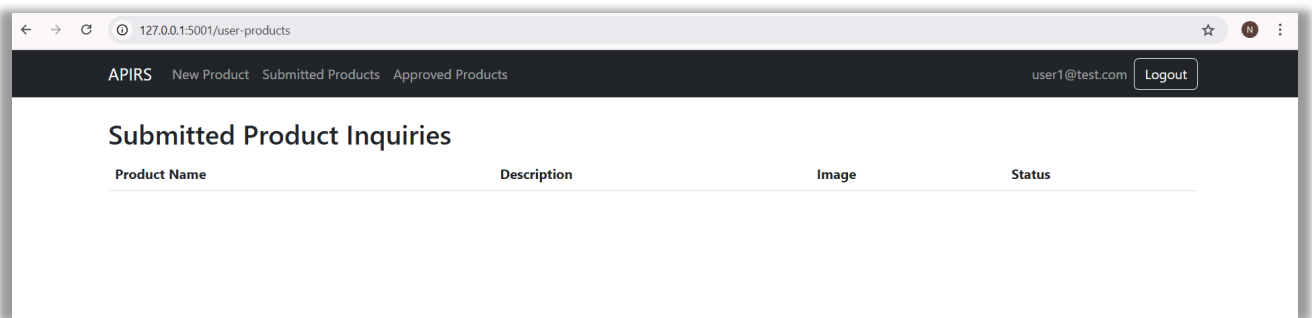
Submitted Products Page – before approval



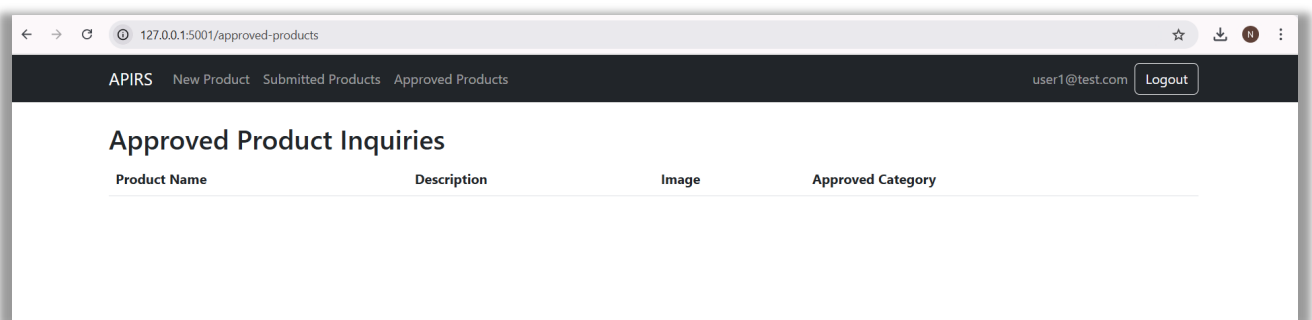
A screenshot of the 'Submitted Product Inquiries' page. The browser address bar shows '127.0.0.1:5001/user-products'. The dark header is identical to the previous page. A green success message 'Product submitted successfully!' is displayed at the top. Below it, the heading 'Submitted Product Inquiries' is followed by a table with four columns: 'Product Name', 'Description', 'Image', and 'Status'.

Product Name	Description	Image	Status
Miniature LED Bulb	A small, low-wattage LED light bulb used for applications requiring compact lighting solutions.		Pending

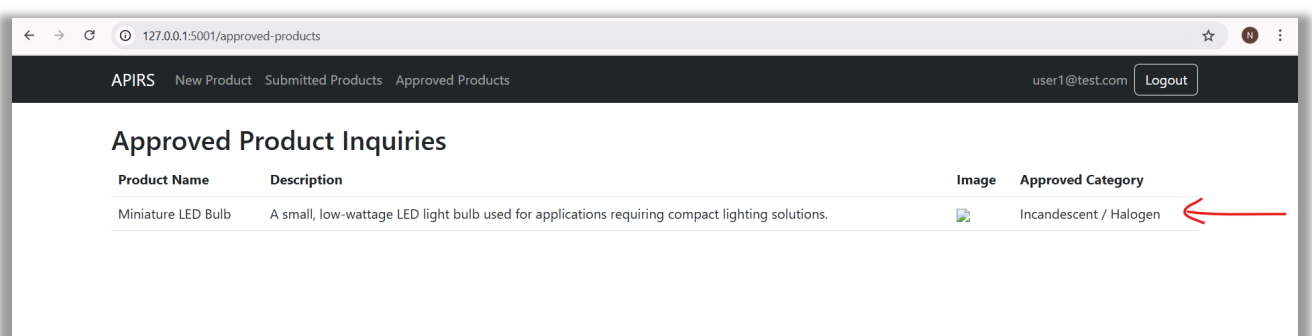
Submitted Products Page – after approval



Approved Products Page – before approval

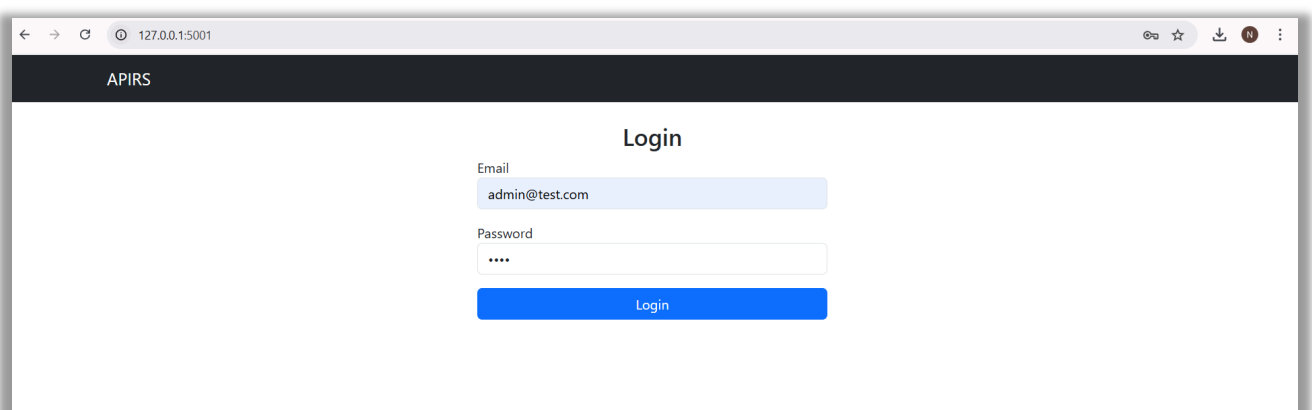


Approved Products Page – after approval

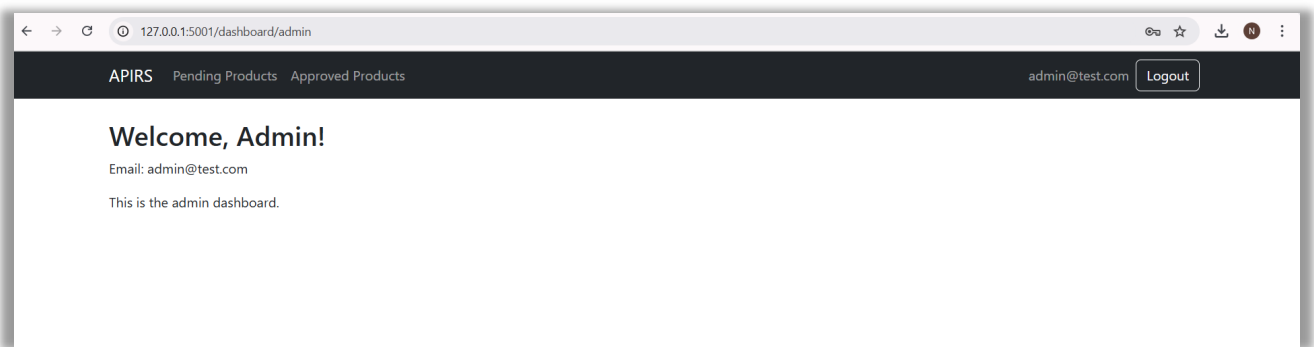


Frontend – UI (admin portal)

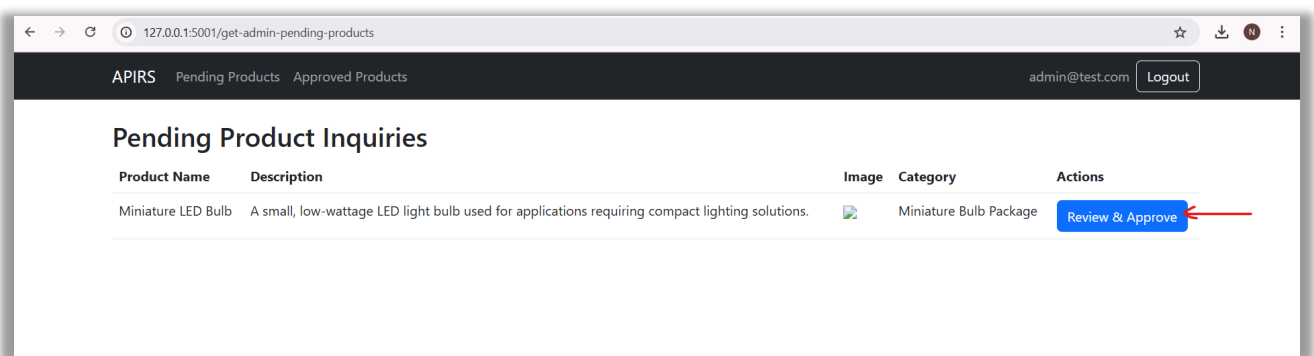
Admin Login Page



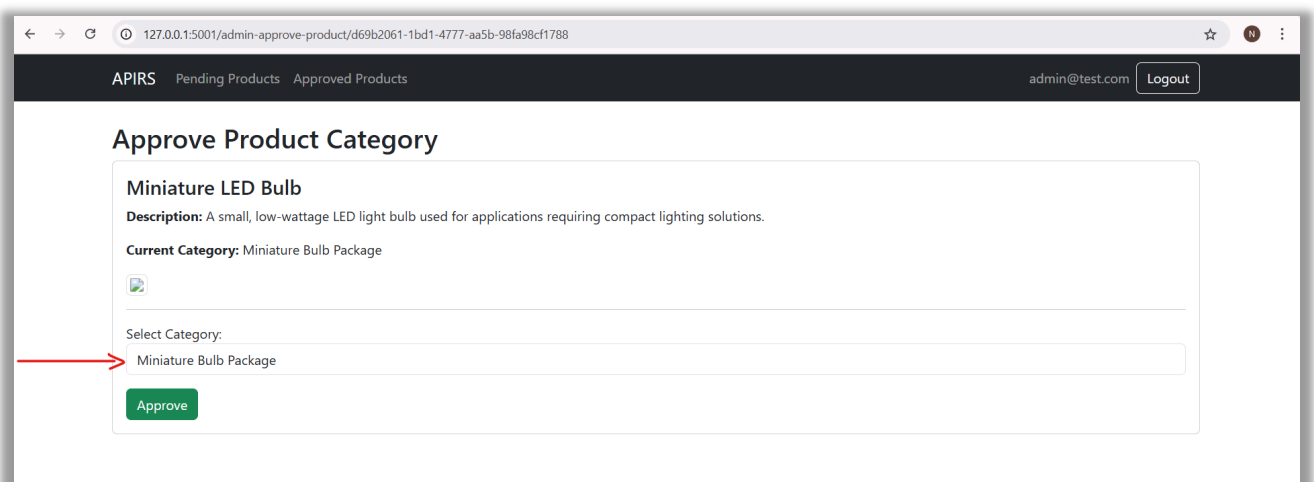
Admin Landing Page



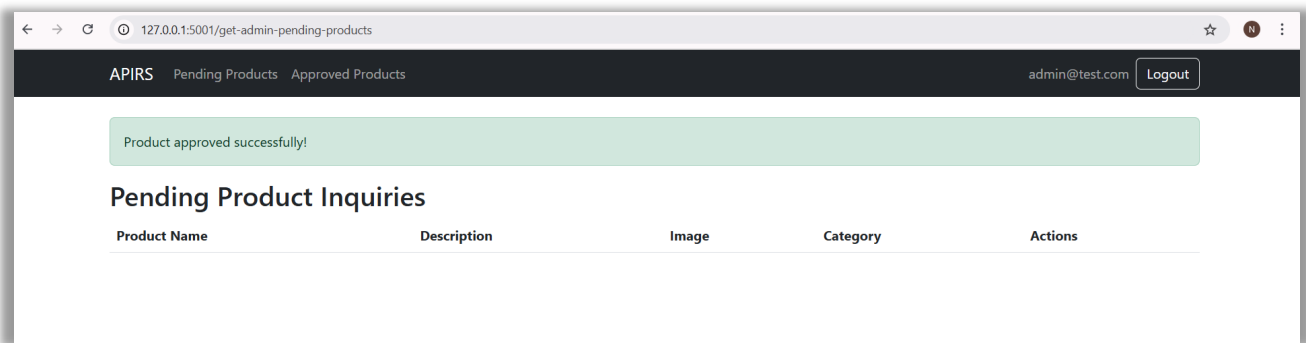
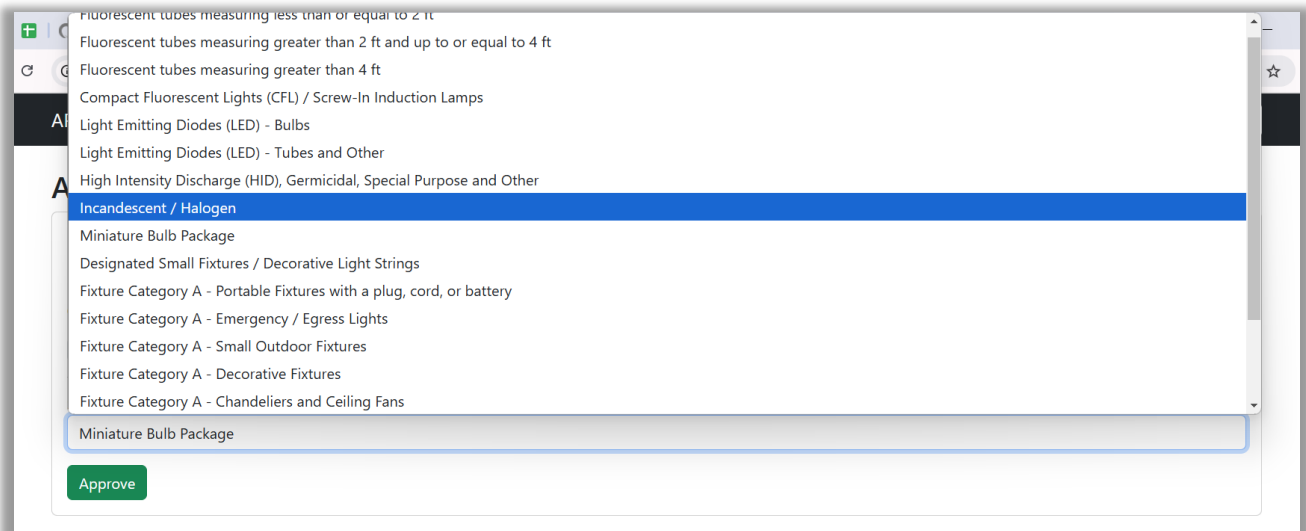
Pending Products Page



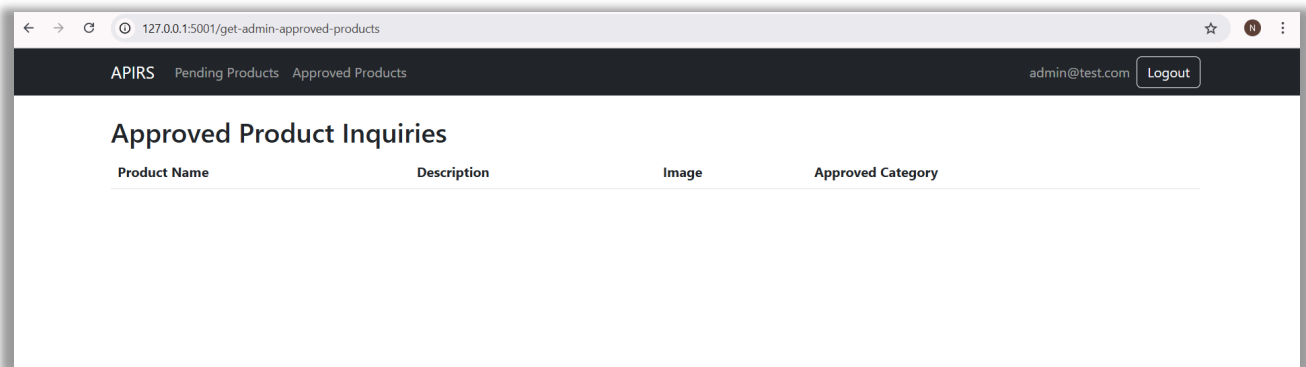
Approve Product Page



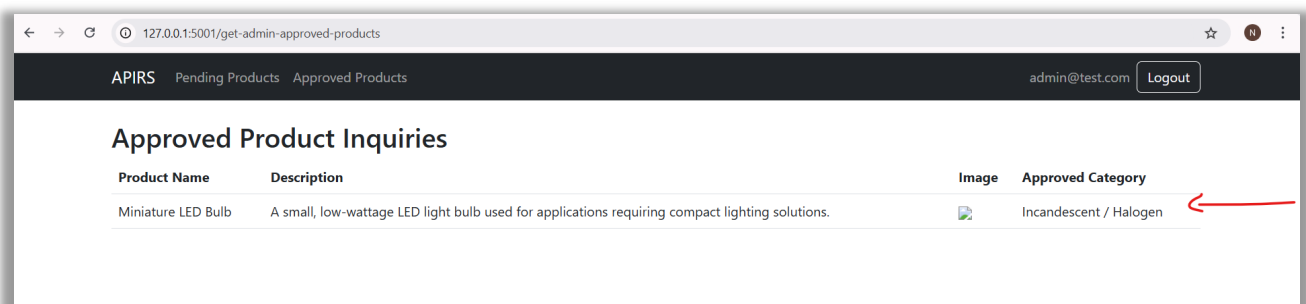
Look (above image) at how the best-matched category has been already selected in the Select Category drop down list. If the admin is not satisfied with the ML model's suggestion, he/she can override it by manually selecting the category from the drop down (below image) and then approve.



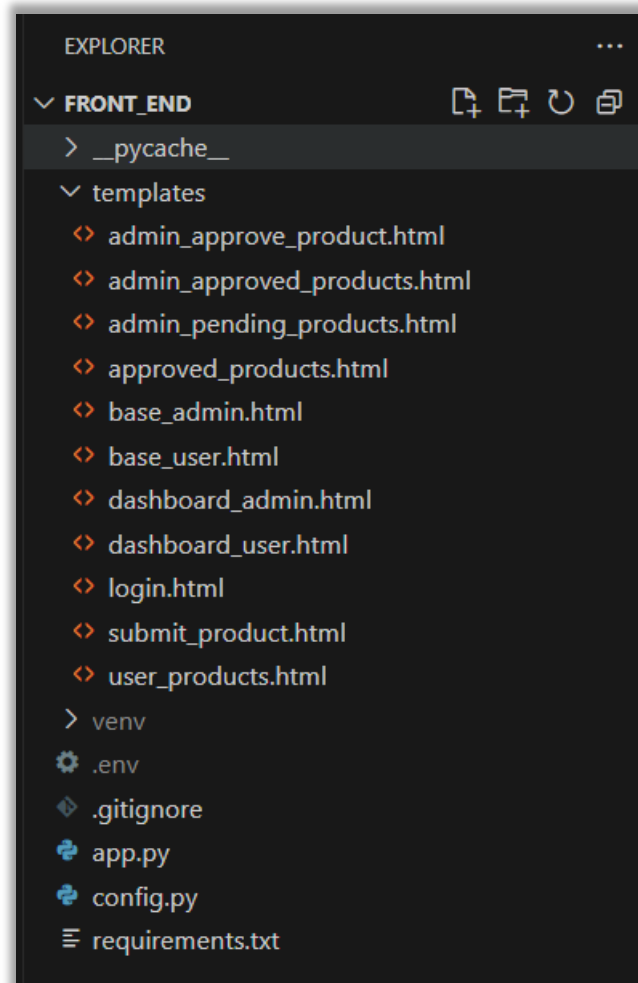
Approved Products Page – before approval



Approved Products Page – after approval

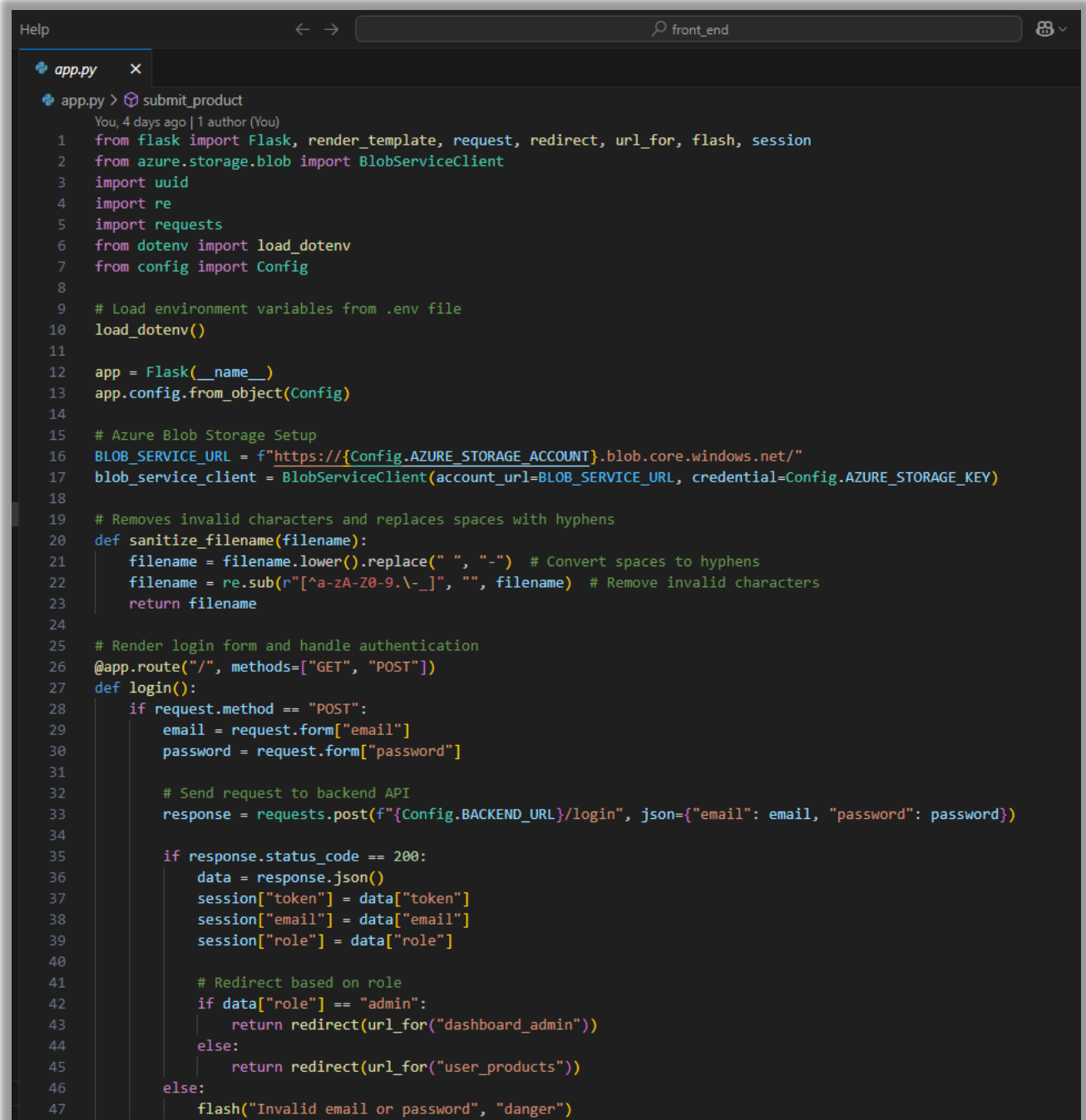


Frontend – Code



```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  GITLENS
PS C:\Users\names\W25_4495_S2_NamesH\Implementation\front_end> venv\Scripts\activate
(venv) PS C:\Users\names\W25_4495_S2_NamesH\Implementation\front_end> python app.py
* Serving Flask app 'app'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5001
Press CTRL+C to quit
* Restarting with stat
* Debugger is active!
* Debugger PIN: 149-390-833
127.0.0.1 - - [24/Feb/2025 14:20:00] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [24/Feb/2025 14:21:48] "POST / HTTP/1.1" 302 -
127.0.0.1 - - [24/Feb/2025 14:21:49] "GET /user-products HTTP/1.1" 200 -
127.0.0.1 - - [24/Feb/2025 14:22:45] "GET /user-products HTTP/1.1" 200 -
127.0.0.1 - - [24/Feb/2025 14:23:08] "GET /approved-products HTTP/1.1" 200 -
127.0.0.1 - - [24/Feb/2025 14:23:30] "GET /user-products HTTP/1.1" 200 -
```

app.py

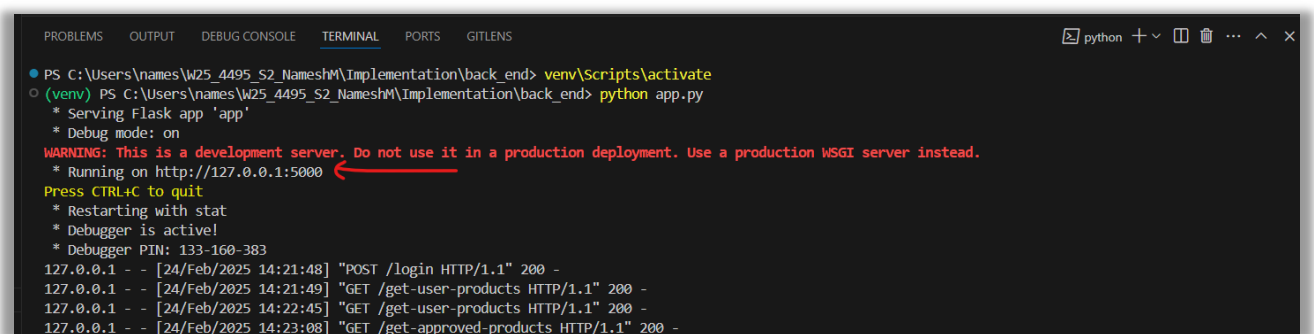
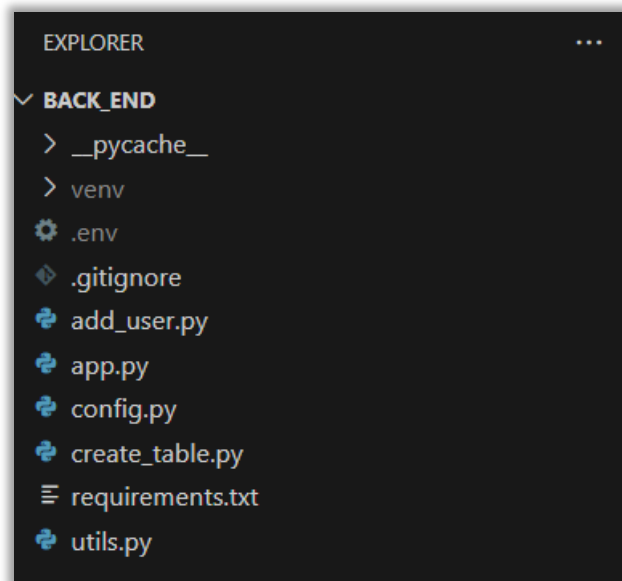


```

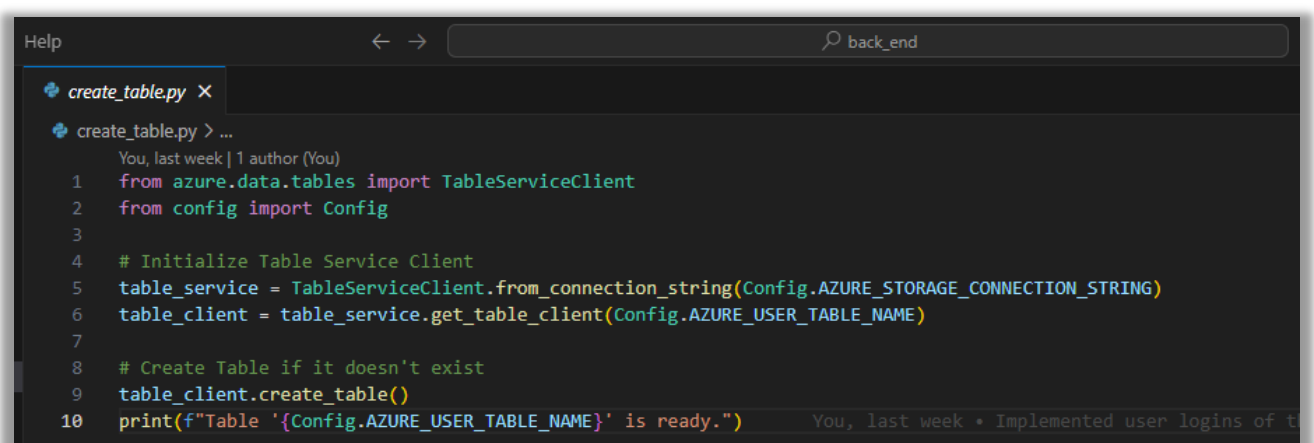
1  from flask import Flask, render_template, request, redirect, url_for, flash, session
2  from azure.storage.blob import BlobServiceClient
3  import uuid
4  import re
5  import requests
6  from dotenv import load_dotenv
7  from config import Config
8
9  # Load environment variables from .env file
10 load_dotenv()
11
12 app = Flask(__name__)
13 app.config.from_object(Config)
14
15 # Azure Blob Storage Setup
16 BLOB_SERVICE_URL = f"https://{Config.AZURE_STORAGE_ACCOUNT}.blob.core.windows.net/"
17 blob_service_client = BlobServiceClient(account_url=BLOB_SERVICE_URL, credential=Config.AZURE_STORAGE_KEY)
18
19 # Removes invalid characters and replaces spaces with hyphens
20 def sanitize_filename(filename):
21     filename = filename.lower().replace(" ", "-") # Convert spaces to hyphens
22     filename = re.sub(r"[^a-zA-Z0-9.\-_]", "", filename) # Remove invalid characters
23     return filename
24
25 # Render login form and handle authentication
26 @app.route("/", methods=["GET", "POST"])
27 def login():
28     if request.method == "POST":
29         email = request.form["email"]
30         password = request.form["password"]
31
32         # Send request to backend API
33         response = requests.post(f"{Config.BACKEND_URL}/login", json={"email": email, "password": password})
34
35         if response.status_code == 200:
36             data = response.json()
37             session["token"] = data["token"]
38             session["email"] = data["email"]
39             session["role"] = data["role"]
40
41             # Redirect based on role
42             if data["role"] == "admin":
43                 return redirect(url_for("dashboard_admin"))
44             else:
45                 return redirect(url_for("user_products"))
46         else:
47             flash("Invalid email or password", "danger")

```

Backend – Code



create_table.py



add-user.py

```

Help
← → back_end

add_user.py X
add_user.py > ...
You, 7 days ago | 1 author (You)
1 import uuid          You, last week * Implemented user logins of the web portal
2 from azure.data.tables import TableServiceClient, TableEntity
3 from config import Config
4 import bcrypt
5
6 # Azure Table connection
7 table_service = TableServiceClient.from_connection_string(Config.AZURE_STORAGE_CONNECTION_STRING)
8 user_table = table_service.get_table_client(Config.AZURE_USER_TABLE_NAME)
9
10 # Generate a bcrypt hash for a given password
11 def hash_password(password):
12     return bcrypt.hashpw(password.encode("utf-8"), bcrypt.gensalt()).decode("utf-8")
13
14 # Insert a user into Azure Table Storage
15 def add_user(email, password):
16     user_id = str(uuid.uuid4()) # Generate unique ID
17     hashed_password = hash_password(password)
18
19     user_entity = TableEntity()
20     user_entity["PartitionKey"] = "users" # Fixed partition key for all users
21     user_entity["RowKey"] = user_id # Unique identifier for each user
22     user_entity["Email"] = email
23     user_entity["PasswordHash"] = hashed_password
24
25     user_table.create_entity(user_entity)
26     print(f"User {email} added successfully!")
27
28 # Run this script with example user
29 if __name__ == "__main__":
30     email = input("Enter email: ")
31     password = input("Enter password: ")
32     add_user(email, password)

```

utils.py

```

Help
← → back_end

utils.py X
utils.py > verify_password
You, 7 days ago | 1 author (You)
1 import bcrypt
2
3 # Generate a bcrypt hash for a given password
4 def hash_password(password):
5     return bcrypt.hashpw(password.encode("utf-8"), bcrypt.gensalt()).decode("utf-8")
6
7 # Verify a password against a stored hash
8 def verify_password(stored_password, provided_password):
9     return bcrypt.checkpw(provided_password.encode("utf-8"), stored_password.encode("utf-8"))

```

app.py

```

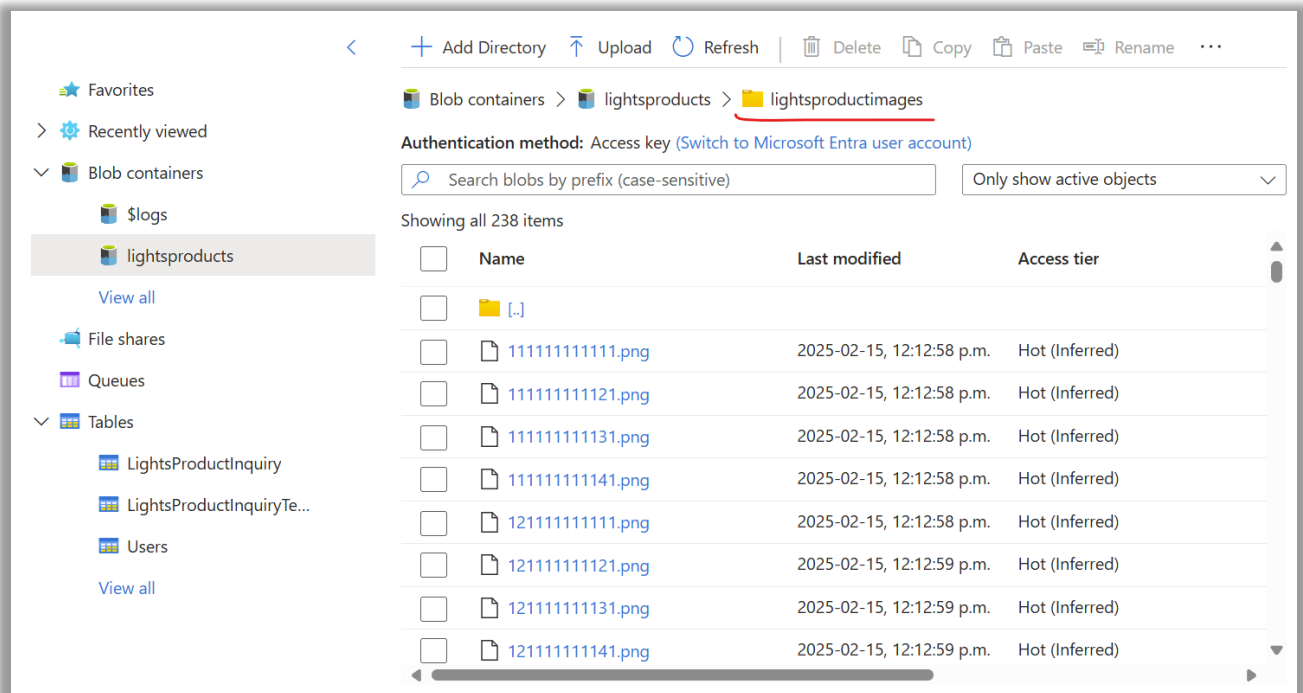
Help  ← →  🔍 back_end

app.py x
app.py > submit_product
You, 4 days ago | 1 author (You)
1  from flask import Flask, request, jsonify
2  from azure.data.tables import TableServiceClient, TableEntity
3  from flask_jwt_extended import JWTManager, jwt_required, get_jwt_identity, create_access_token
4  from werkzeug.security import check_password_hash
5  import uuid
6  import requests
7  import json
8
9  from config import Config
10 from utils import verify_password
11
12 app = Flask(__name__)
13 app.config.from_object(Config) # Load configuration from config.py
14
15 jwt = JWTManager(app)
16
17 # Azure Table Storage connection
18 table_service = TableServiceClient.from_connection_string(Config.AZURE_STORAGE_CONNECTION_STRING)
19 user_table = table_service.get_table_client(Config.AZURE_USER_TABLE_NAME)
20 product_table = table_service.get_table_client(Config.AZURE_TEMP_PRODUCT_TABLE_NAME)
21
22 # Handle user login
23 @app.route("/login", methods=["POST"])
24 def login():
25     data = request.get_json()
26     email = data.get("email")
27     password = data.get("password")
28
29     if not email or not password:
30         return jsonify({"error": "Email and password are required"}), 400
31
32     # Query user by email
33     try:
34         query = f"Email eq '{email}'"
35         users = list(user_table.query_entities(query))
36
37         if not users:
38             return jsonify({"error": "Invalid email or password"}), 401
39
40         user = users[0] # First matching user
41         stored_password = user["PasswordHash"]
42
43         # Verify password
44         if not verify_password(stored_password, password):
45             return jsonify({"error": "Invalid email or password"}), 401
46
47         # Determine user role

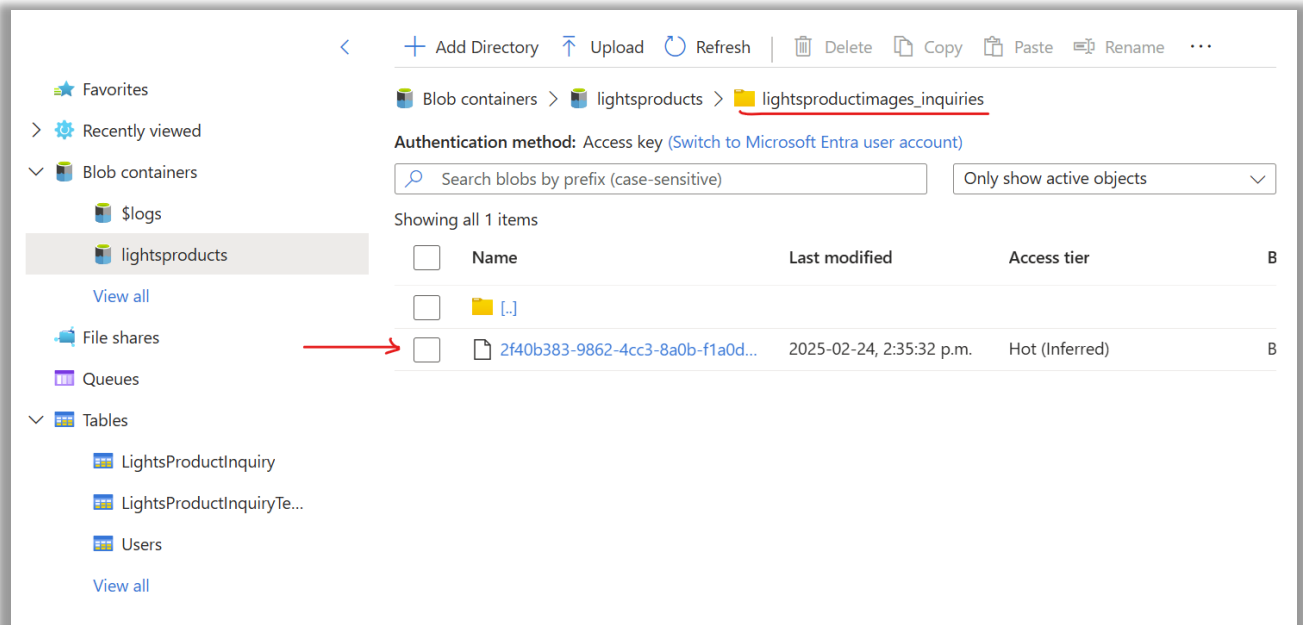
```

Azure Blob Storage and Azure Table Storage

lightsproductimages – Azure Blob Storage that stores the images of the products in the product guide



lightsproductimages_inquiries – Azure Blob Storage that stores the images of the products in the new product inquiries



Users - Azure Table that stores the details of the users of the web portal

[Add entity](#) [Refresh](#) [Delete](#) [Edit columns](#)

[Tables](#) > [Users](#)

Authentication method: Access key ([Switch to Microsoft Entra user account](#))

[Add filter](#) Advanced filters

Showing all 2 items

☐	PartitionKey	RowKey	Timestamp	Email
☐	users	bc2c473c-b42b-4732-8...	2025-02-17T22:20:23.35...	user1@test.com
☐	users	f380ec04-e53a-4514-a4...	2025-02-17T22:19:58.69...	admin@test.com

LightsProductInquiry - Azure Table that stores the textual data of the products in the product guide

[Add entity](#) [Refresh](#) [Delete](#) [Edit columns](#)

[Tables](#) > [LightsProductInquiry](#)

Authentication method: Access key ([Switch to Microsoft Entra user account](#))

[Add filter](#) Advanced filters

Showing all 294 items

☐	PartitionKey	RowKey	Timestamp	british_columbia
☐	Alarms, phones, and dev...	a525147b-2ce0-48a1-b...	2025-02-18T08:04:48.19...	
☐	Aquarium equipment	28362b34-5950-4574-a...	2025-02-18T08:04:48.25...	
☐	Auto fixtures	29bbdf85-db9e-4698-9...	2025-02-18T08:04:48.31...	
☐	Backlit signs	27baf64d-92b8-42f6-a4...	2025-02-18T08:04:48.37...	
☐	Black light equipment	829934ce-8cf7-4b61-b9...	2025-02-18T08:04:48.50...	
☐	Bug zappers	dca6f1c1-4f70-439e-b3...	2025-02-18T08:04:48.56...	
☐	Camera and video acces...	f994b44e-182e-412f-a8...	2025-02-18T08:04:48.62...	
☐	Compact Fluorescent Li...	141111111111	2025-02-18T08:04:33.16...	0.15

LightsProductInquiryTemp - Azure Table that stores the textual data of the new product inquiries

The screenshot shows the Azure Portal interface for the 'LightsProductInquiryTemp' table. The left sidebar contains a navigation menu with 'Tables' expanded, showing 'LightsProductInquiryTemp' selected. The main content area displays the table's details, including the authentication method (Access key) and a list of items. A red arrow points to the table name in the sidebar, and another red arrow points to the table name in the main content area.

PartitionKey	RowKey	Timestamp	ProductCategory
bc2c473c-b42b-4732-8...	d69b2061-1bd1-4777-a...	2025-02-24T22:35:38.05...	Miniature Bulb Pack

Newly submitted product inquiry in LightsProductInquiryTemp – before approval

Property Name	Type	Value
PartitionKey	String	bc2c473c-b42b-4732-8013-3fa95e78e092
RowKey	String	d69b2061-1bd1-4777-aa5b-98fa98cf1788
Timestamp	DateTi...	2025-02-24T22:35:38.0594744Z
ProductCategory	String	Miniature Bulb Package
ProductDescription	String	A small, low-wattage LED light bulb used for application...
ProductImageUrl	String	
ProductName	String	Miniature LED Bulb
Status	String	Pending

app.py

```

1  from flask import Flask, request, jsonify
2  from azure.data.tables import TableServiceClient
3  from sentence_transformers import SentenceTransformer, util
4  import numpy as np
5  from config import Config
6  from azure.core.credentials import AzureNamedKeyCredential
7
8
9  app = Flask(__name__)
10 app.config.from_object(Config)
11
12 # Create credential object
13 credential = AzureNamedKeyCredential(Config.AZURE_STORAGE_ACCOUNT_NAME, Config.AZURE_STORAGE_ACCOUNT_KEY)
14
15 # Azure Storage URLs
16 BLOB_SERVICE_URL = f"https://{Config.AZURE_STORAGE_ACCOUNT_NAME}.blob.core.windows.net/"
17 TABLE_SERVICE_URL = f"https://{Config.AZURE_STORAGE_ACCOUNT_NAME}.table.core.windows.net/"
18
19 # Load NLP model
20 model = SentenceTransformer('all-MiniLM-L6-v2')
21
22 # Method to fetch product data from Azure Table Storage
23 def fetch_products_from_azure():
24     try:
25         service = TableServiceClient(endpoint=TABLE_SERVICE_URL, credential=credential)
26         table_client = service.get_table_client(table_name=Config.AZURE_TABLE_NAME)
27
28         products = []
29         entities = table_client.list_entities()
30
31         for entity in entities:
32             products.append({
33                 "product_category": entity["PartitionKey"],
34                 "product_id": entity.get("product_id", ""),
35                 "product_description": entity.get("product_description", ""),
36                 "row_key": entity["RowKey"]
37             })
38
39         return products
40
41     except Exception as e:
42         print(f"Error fetching products from Azure Table Storage: {e}")
43         return []
44
45 # Method to compute similarity using NLP
46 def find_best_match(new_product_name, new_product_description, products):
47     product_texts = [f"{p['product_category']} {p['product_description']}" for p in products]

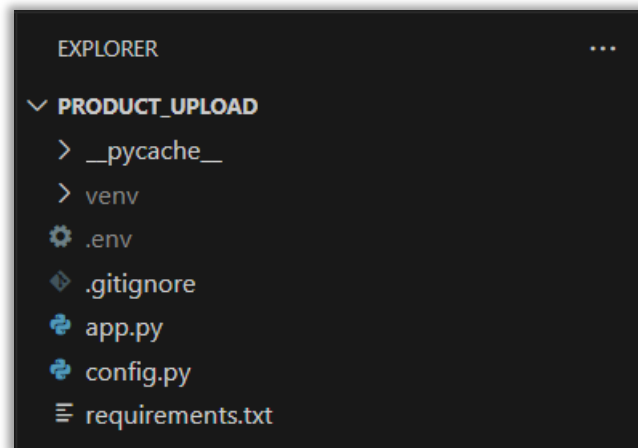
```

Product Upload – Code

```

PS C:\Users\names\W25_4495_S2_NameshM\Implementation\product_upload> .venv\Scripts\activate
(venv) PS C:\Users\names\W25_4495_S2_NameshM\Implementation\product_upload> python app.py
Downloading Excel file...
Filtering and preparing data...
Total rows to upload: 294
Deleting existing records from Azure Table Storage...
Deleted 294 existing records successfully.
Uploading data to Azure Table Storage...
Data upload complete.
(venv) PS C:\Users\names\W25_4495_S2_NameshM\Implementation\product_upload>

```



app.py

```

Help
← → product_upload
app.py x
app.py > main
You, 7 days ago | 1 author (You)
1 import io
2 import pandas as pd
3 import uuid
4 from azure.storage.blob import BlobServiceClient
5 from azure.data.tables import TableServiceClient
6 from dotenv import load_dotenv
7 from azure.core.credentials import AzureNamedKeyCredential
8 from config import config
9 from flask import Flask
10
11 app = Flask(__name__)
12 app.config.from_object(config)
13
14 # Create credential object
15 credential = AzureNamedKeyCredential(config.AZURE_STORAGE_ACCOUNT_NAME, config.AZURE_STORAGE_ACCOUNT_KEY)
16
17 # Azure Storage URLs
18 BLOB_SERVICE_URL = f"https://{config.AZURE_STORAGE_ACCOUNT_NAME}.blob.core.windows.net/"
19 TABLE_SERVICE_URL = f"https://{config.AZURE_STORAGE_ACCOUNT_NAME}.table.core.windows.net/"
20
21 # Initialize Azure clients
22 blob_service_client = BlobServiceClient(account_url=BLOB_SERVICE_URL, credential=config.AZURE_STORAGE_ACCOUNT_KEY)
23 table_service_client = TableServiceClient(endpoint=TABLE_SERVICE_URL, credential=credential)
24 table_client = table_service_client.get_table_client(table_name=config.AZURE_TABLE_NAME)
25
26 # Downloads the Excel file from Azure Blob Storage and loads it into a DataFrame
27 def download_excel_from_blob():
28     try:
29         blob_client = blob_service_client.get_blob_client(container=config.AZURE_BLOB_CONTAINER_NAME, blob=config.EXCEL_BLOB_NAME)
30
31         # Check if the blob exists
32         if not blob_client.exists():
33             raise FileNotFoundError(f"Blob {config.EXCEL_BLOB_NAME} not found in container {config.AZURE_BLOB_CONTAINER_NAME}.")
34
35         download_stream = blob_client.download_blob().readall()
36         df = pd.read_excel(io.BytesIO(download_stream), engine='openpyxl')
37
38         return df
39
40     except Exception as e:
41         print(f"Error downloading Excel file from Azure Blob Storage: {e}")
42         return None
43
44 # Filters the required columns, generates unique RowKeys, and adds the image URL column
45 def filter_and_prepare_data(df):
46     # Select only the needed columns and explicitly create a copy
47     df = df[["source", "product_id", "product_category", "product_description",

```

Work Logs

Phase	Date	#	Number of Hours	Tr	Description of Work Done
Midterm Report	10/02/2025	2			Merged 2 product categories (Light Emitting Diodes (LED) - Bulbs & Light Emitting Diodes) into 1 (Light Emitting Diodes (LED) - Bulbs). Both are actually the same.
Midterm Report	11/02/2025	2			Set up Azure Storage Account in the resource group provided by the Company.
Midterm Report	12/02/2025	2			Replaced the existing Images in the products table with high-quality versions.
Midterm Report	13/02/2025	3			Set up Azure Blob Container (to store images) and Azure Table (to store textual data) in the created Storage Account. Added the script to extract images into Azure Blob Storage and then create Azure Table
Midterm Report	14/02/2025	3			Went through tutorials/videos on Flask & Bootstrap
Midterm Report	15/02/2025	4			Went through tutorials/videos on Flask & Bootstrap & Started working on the user logins of the web portal
Midterm Report	16/02/2025	5			Implemented user logins of the web portal & started working on the other components
Midterm Report	17/02/2025	5			Implemented other components of the web portal
Midterm Report	18/02/2025	1			Meeting with PCA staff - Showed the web portal so far and got their feedback
Midterm Report	19/02/2025	2			Went through articles/videos on Text Similarity in NLP
Midterm Report	20/02/2025	2			Implemented ML model API and connection to backend
Midterm Report	21/02/2025	1			Removed venv from Git tracking & added requirements.txt for all the apps
Midterm Report	22/02/2025	1			Researched on computing similarity of texts using NLP
Midterm Report	23/02/2025	4			Implemented the initial stage of the ML Model - in this stage, prediction is based on just matching the 'Product Name' of the new product with 'Product Category' of the existing products in product guide. Also, created the PROJECT_INSTALLATION_INSTRUCTIONS_MIDTERM.md
Midterm Report	24/02/2025	4			Drafted and finalized the midterm report and the demo video

Above are the logs detailing the work completed after submitting Progress Report 1. For a complete view of all work logs, please refer to this [Google Sheet](#).

Acknowledgements

I would like to express my gratitude to Product Care Association (PCA) for providing the data and supporting the development of this project. I also extend my appreciation to my manager, *Luther Trammell*, as well as Zeus Baral, Mario Anda, and Roxanne Yip for their guidance and valuable insights. Special thanks to my instructor, *Padmapriya Arasanipalai Kandhadai*, for her continuous feedback and support.

References

- PCA Official Website: <https://www.productcare.org>
- How are lights and light bulbs recycled: <https://www.youtube.com/watch?v=KHmwDosH-FQ>
- PCA Light Recycling Product Guide (Product Catalogue)
- Flask Documentation (3.1.x): <https://flask.palletsprojects.com/en/stable/>
- Bootstrap-Flask - Bootstrap-Flask 2.2.x documentation - <https://bootstrap-flask.readthedocs.io/en/stable/>
- NLP & BERT-based Text Matching: <https://huggingface.co/transformers/>
- Computer Vision with ResNet: https://pytorch.org/hub/pytorch_vision_resnet/
- FAISS for Similarity Search: <https://faiss.ai/>
- ChatGPT: OpenAI